

Configuring MCP (Model Context Protocol) Tool Access for Local LLM Agents in LM Studio: A Cross-Platform Debugging Case Study

Author: Jerry Abramson

Date: 2026-07-14

Context: Boston University research on local LLM tooling for agentic workflows across heterogeneous hardware (Apple Silicon macOS, Nvidia GB10/Spark ARM devkit, AMD Ryzen AI Strix Halo devkit).

1. Overview

This document records the process of enabling web search tool access for locally-hosted LLMs running in **LM Studio**, via the Model Context Protocol (MCP), across three different machines. What began as a routine configuration task surfaced three independent, stacked failure modes — one specific to macOS GUI application PATH resolution, one an OS-level network privacy permission, and one a latent bug in a personal shell configuration file that silently broke any tool relying on shell-environment introspection. The debugging methodology and root causes are documented here because they generalize beyond this specific setup and are relevant to reproducibility of local-LLM-agent research environments.

2. Motivation

For agentic workflows that require web search (e.g., retrieval-augmented tasks, fact-checking, current-events queries), a locally hosted LLM needs a tool-use mechanism. LM Studio supports MCP, the same tool-integration protocol used by Claude Desktop and other agent frontends. **SearXNG**, a self-hosted metasearch engine, was already running and reachable at `http://gb10.jabramson.tech:8080` on the local network. The `mcp-searxng` npm package (an MCP server that wraps SearXNG's search API) was chosen as the tool-access layer, run via `npm` rather than a container, to minimize the runtime dependency surface for what is otherwise a lightweight process.

The goal: get a `searxng` MCP tool working identically in LM Studio on three research machines:

Machine	Role	OS / Arch	Shell
macOS laptop	primary dev machine	macOS, arm64	zsh (login), bash (nvm)
gb10 (gb10.jabramson.tech)	Nvidia GB10 "Spark" devkit	Linux, aarch64	bash
amd-halo	AMD Ryzen AI "Strix Halo" devkit	Linux, x86_64	bash

3. Initial Configuration and First Failure

LM Studio reads MCP server definitions from `mcp.json` (`~/ .lmstudio/mcp.json` on macOS, `~/ .lmstudio/mcp.json` on the Linux machines), using the same `mcpServers` schema as Claude Desktop:

```
{
  "mcpServers": {
    "searxng": {
      "command": "npx",
      "args": ["-y", "mcp-searxng"],
      "transport": "stdio",
      "env": {
        "SEARXNG_URL": "http://gb10.jabramson.tech:8080"
      }
    }
  }
}
```

On macOS, enabling this server in LM Studio's UI produced an indefinite **"Initializing plugin"** state that never resolved.

4. Root Cause #1 (macOS): GUI Applications Do Not Inherit

Shell PATH

Finding: The user's `node / npm / npx` toolchain on macOS was installed exclusively via `nvm`, at `/Users/jerry/.nvm/versions/node/v23.6.0/bin/`. This directory is added to `PATH` only by interactive shell startup files (`.zshrc / .bashrc`). macOS GUI applications launched from Finder/Dock/Spotlight — including Electron apps like LM Studio — do **not** source shell rc files, and instead receive a minimal default `PATH` (`/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin`, plus `/etc/paths.d/*` entries).

Verification was done by simulating the GUI PATH directly:

```
env -i PATH="/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin" which node
# no output – node not found
```

`/usr/local/bin/npx` on this system was itself a symlink into a Homebrew-managed `npm` install's `npx-cli.js`, a script with a `#!/usr/bin/env node` shebang — so even the symlink target required `node` to be resolvable via `PATH` at the moment the shebang was interpreted, which it was not.

A second, subtler layer: even invoking `node <path-to-npx-cli.js>` directly bypasses the *outer* shebang, but `npx` itself then re-spawns the target package's own bin script (`mcp-searxng`), which has its **own** `#!/usr/bin/env node` shebang — so the fix had to ensure `node` was resolvable via `PATH` for the *entire* process tree, not just the initial command.

Fix: Set the MCP server's `env.PATH` explicitly to include the `nvm` Node's `bin` directory, and point `command` at that Node's `npx` via absolute path:

```
{
  "mcpServers": {
    "searxng": {
      "command": "/Users/jerry/.nvm/versions/node/v23.6.0/bin/npx",
      "args": ["-y", "mcp-searxng"],
      "transport": "stdio",
      "env": {
        "SEARXNG_URL": "http://gb10.jabramson.tech:8080",
        "PATH": "/Users/jerry/.nvm/versions/node/v23.6.0/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"
      }
    }
  }
}
```

```
}  
}  
}  
}
```

This was verified in isolation with `env -i PATH="<minimal-path>" npx -y mcp-searxng`, confirming the server connects and emits its STDIO startup banner in under a second once the PATH included the correct Node directory.

5. Root Cause #2 (macOS): Local Network Privacy Permission

Even after the PATH fix, the plugin still failed, though the symptom changed from an indefinite hang to a hard **"plugin initialization timed out"** error after ~60 seconds.

Finding: `gb10.jabramson.tech` resolves to a private LAN address (`192.168.1.209`). Since macOS Big Sur, connections from an application to devices on the local network are gated behind a distinct **Local Network** privacy permission (System Settings → Privacy & Security → Local Network), separate from general internet access. Checking this setting confirmed LM Studio was listed but **toggled off** — meaning any local-network connection attempt made by LM Studio or its child processes would be blocked at the OS level.

Fix: Enable the Local Network toggle for LM Studio, then fully quit and relaunch the application (the permission is evaluated at process start).

6. Root Cause #3 (macOS): A Personal Shell Configuration Bug Breaking Non-Interactive Shell Invocation

After both fixes above, the timeout **persisted unchanged** — same exact ~60-second window, regardless of server (it affected an unrelated, previously-configured `filesystem` MCP server identically). This ruled out both prior explanations, since the process was never even observed to spawn (`ps` polling across multiple live retry attempts, synchronized with LM Studio's log timestamps, never showed an `npx / mcp-searxng` process during the entire failure window).

Finding: Static analysis of LM Studio's bundled application code (`mcpbridgeworker.js`) revealed

that before spawning a configured MCP server, LM Studio attempts to **capture the user's real shell environment** (a common technique in Electron/GUI applications, used specifically to work around the PATH problem in §4) — with a literal fallback string "Failed to get shell environment, using process.env" in the code, indicating it invokes the user's login shell non-interactively (functionally equivalent to `$SHELL -ilc '...'`) to capture environment variables such as `PATH`.

The user's `~/.zshrc` contained:

```
if [ "$SHELL" = "/bin/zsh" -o "$TERM_PROGRAM" = "Termius" ]; then
    if [ -x /opt/local/bin/bash ]; then
        exec /opt/local/bin/bash --login
    fi
else
    exec /bin/bash --login
fi
```

This unconditionally `exec`s into a bare, interactive login `bash` whenever `zsh` starts with `$SHELL=/bin/zsh` — a personal convenience redirect (originally intended to normalize the interactive terminal experience). Critically, `exec` **replaces the process image outright**, discarding any command passed via `zsh`'s `-c` flag. Any tool — not just LM Studio — that shells out via `zsh -c '...'` or `zsh -ilc '...'` to introspect the environment would have its intended command silently dropped, landing instead in a bare interactive `bash --login` session that sits reading from `stdin` indefinitely. When such a caller does not close `stdin` immediately, this manifests as a hang bounded only by whatever timeout the calling application enforces — in LM Studio's case, ~60 seconds.

This was confirmed empirically:

```
$ time /bin/zsh -ilc 'echo SHELL_ENV_OK'
[Using Native macOS bash via /bin/zsh]
# "SHELL_ENV_OK" never printed - command was silently dropped
```

Fix: `zsh` sets `$ZSH_EXECUTION_STRING` internally whenever it is invoked with an explicit `-c` command. Guarding the redirect on the *absence* of this variable preserves the original interactive-terminal behavior while allowing non-interactive, command-bearing invocations to proceed normally:

```
if [ "$STARTING_X" != "true" ] && [ -z "$ZSH_EXECUTION_STRING" ]; then
    # ... original redirect logic, now only reached for genuinely
    # interactive shells with no explicit command ...
fi
```

Verified post-fix:

```
$ time /bin/zsh -ilc 'echo SHELL_ENV_OK; env | wc -l' </dev/null
SHELL_ENV_OK
111
real    0m0.698s
```

After this fix, LM Studio's MCP plugin connected successfully within seconds.

Broader significance: this bug was not LM-Studio-specific — it would silently break *any* tool relying on `zsh -c / zsh -ilc` to introspect the user's environment (a pattern also used by some other Electron apps, e.g. VS Code's `fix-path`-style shell resolution). Its root cause lived entirely in a personal dotfile, invisible to the affected application's own diagnostics, and produced a symptom (a fixed-duration timeout with zero subprocess evidence) that pointed away from the actual defect until process-spawn observation (via synchronized `ps` polling and LM Studio's internal log timestamps) ruled out the two more "obvious" causes.

7. Cross-Platform Replication: `gb10` and `amd-halo`

Having a working macOS reference configuration, the same `searxng` MCP tool was deployed to the two Linux devkits. Each had its own, different obstacle — underscoring that "the same config" does not mean "the same fix" across heterogeneous environments.

7.1 `gb10` (Nvidia GB10/Spark, aarch64)

- Was previously running the MCP server via Docker (`docker run --rm isokoliuk/mcp-searxng`), chosen specifically because the system's package-managed Node.js was v18.19.1, while `mcp-searxng` requires Node ≥ 20 (confirmed via a direct failure: `ReferenceError: File`

is not defined inside `undici`, a dependency requiring Node's native `File` global introduced in v20).

- Rather than modify the system Node install (used elsewhere) or introduce a version manager (which, per §4 and §6, adds its own PATH-resolution risk surface), a self-contained Node v24.18.0 LTS (arm64) tarball was downloaded directly from nodejs.org and extracted to `/home/jerry/.local/node-v24.18.0/`, with no shell integration.
- `mcp.json` was updated to invoke this Node's `npx` by absolute path, eliminating the Docker dependency.
- Shell startup files were checked for the class of bug in §6; none was present (default shell is plain bash with no `exec` redirects active).

7.2 amd-halo (AMD Ryzen AI "Strix Halo", x86_64)

- Had no existing MCP configuration (`mcpServers: {}`).
- System Node was v20.19.2 (version-adequate), but the Debian `nodejs` package installed did **not** include `npm`/`npx` — only non-functional `corepack` shims were present at `/usr/share/nodejs/corepack/shims/`.
- The same self-contained Node v24.18.0 LTS (x64) approach used for `gb10` was applied here, installed to `/home/jerry/.local/node-v24.18.0/`, sidestepping the incomplete system package.
- No shell-startup redirect bugs were found.

7.3 Final Configuration Pattern (all three machines)

```
{
  "mcpServers": {
    "searxng": {
      "command": "<absolute-path-to-node-install>/bin/npx",
      "args": ["-y", "mcp-searxng"],
      "transport": "stdio",
      "env": {
        "SEARXNG_URL": "http://gb10.jabramson.tech:8080",
        "PATH": "<absolute-path-to-node-install>/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"
      }
    }
  }
}
```

```
}
```

8. Methodology Notes

The debugging approach that ultimately isolated the (non-obvious) third root cause relied on:

1. **Falsifiable, timestamped evidence over speculation.** Each hypothesis (PATH, Local Network permission, sandboxing) was tested by direct reproduction (`env -i PATH=... npx ...`) rather than assumed correct from documentation or plausibility alone.
2. **Correlating application logs with live process observation.** LM Studio's own server logs recorded exact `Client created` / `Client disconnected` timestamps; synchronizing a `ps` polling loop with a live retry in LM Studio's UI established, empirically, that the target subprocess never spawned — which eliminated entire classes of otherwise-plausible explanations (e.g., the SearXNG connection itself failing) and redirected investigation to LM Studio's *pre-spawn* logic.
3. **Reading the actual (minified) application code** when documentation and log output were insufficient — locating the literal fallback string `"Failed to get shell environment, using process.env"` was the pivot point that identified the shell-environment-capture step as the actual point of failure.
4. **Not assuming a fix is complete because a plausible cause was found and addressed.** Two real, independently-fixable issues (§4, §5) were identified and corrected first, and both were necessary — but neither was sufficient, and the timeout symptom was unchanged until the third cause (§6) was found. Each fix was verified in isolation before moving on.

9. Relevance to the Research

This case study is included in the project's documentation because it captures a reproducible environment-configuration procedure for enabling MCP-based tool access in locally-hosted LLM agents across the three heterogeneous hardware targets used in this research (Apple Silicon, Nvidia GB10, AMD Strix Halo). Future environment setup — or replication by collaborators — on any of these platform types can reference §4–§7 directly rather than re-deriving the same fixes.

10. Cost of the Cloud-Model Debugging Effort

The entire debugging effort documented in §3–§7 — across all three machines — was carried out via

Claude Code, using Claude models as the reasoning/tool-use layer. Reported session cost:

Metric	Value
Total cost	\$5.41
Total duration (API/compute)	18m 11s
Total duration (wall clock)	1h 18m 41s
Code/config changes	261 lines added, 7 lines removed

Usage by model:

Model	Input tokens	Output tokens	Cache read	Cache write	Cost
claude-opus-4-8	3.7k	4.9k	133.4k	23.6k	\$0.44
claude-haiku-4-5	536	15	0	0	\$0.0006
claude-sonnet-5	10.9k	70.0k	10.4M	127.7k	\$4.96

Note the large gap between wall-clock time (~79 minutes) and API/compute time (~18 minutes): most of the wall-clock duration was spent waiting on live, synchronized reproduction steps (timed `ps` polling against LM Studio retry attempts, `sleep`-gated background verification of long-running stdio processes) rather than on model inference itself. This distinction matters for §11.

11. Local vs. Cloud Model Tradeoffs: This Session as a Data Point

This project's broader research question — the tradeoffs between locally-hosted models and cloud frontier models — is not just answered by this case study's *subject matter* (getting local models tool-use access via MCP); the debugging process itself is a data point in that comparison.

The setup/debugging cost is real and was paid in cloud-model tokens, not local compute.

Enabling web-search tool access for the local models on these three machines required diagnosing three independent, non-obvious root causes (§4–§6): a GUI-application PATH resolution gap, an OS-level network privacy permission, and a latent bug in a personal dotfile that silently broke shell-environment introspection for *any* consuming tool. None of these live at the model layer — they are infrastructure/OS-level failures that happen to sit between "a local model" and "a local model that can actually use tools." Diagnosing them required: static analysis of a minified, obfuscated Electron application bundle to locate a single load-bearing fallback string; correlating application log timestamps against live, synchronized process-monitoring loops to rule out competing hypotheses; and revising the working theory twice after each prior fix proved necessary but insufficient. This is the kind of multi-hop, evidence-gathering-under-uncertainty reasoning that, at the time of writing, was performed here by cloud frontier models (primarily `claude-sonnet-5`, with `claude-opus-4-8` handling higher-level orchestration) — at a measured cost of \$5.41 and 18 minutes of API compute (§10).

This cost is non-recurring, unlike per-query local inference cost. Once fixed, the `.zshrc` bug, the PATH configuration, and the Local Network permission do not need to be re-diagnosed; all subsequent local-model tool-use on these three machines amortizes this one-time cost to effectively zero. This is a structurally different cost profile than the per-token cost of local inference (bounded by owned hardware and electricity) or cloud inference (bounded by per-token pricing) — it is a *capability bootstrapping* cost, paid once, in a currency (frontier-model reasoning tokens) that is distinct from either.

An open question this raises for the research: could a locally-hosted model (e.g., a 30B-class model run on the GB10 or Strix Halo devkits) have performed this same diagnostic chain autonomously — given the same tool access (shell, log access, live process monitoring) this session used? The chain required holding three sequentially-falsified hypotheses in context, reading and pattern-matching against obfuscated third-party source code, and synchronizing real-time observations against application logs over a debugging session spanning many tool-call rounds. This is a concrete, reproducible task (the artifacts — logs, the exact commands run, the exact code excerpts inspected — are all in §4–§6 above) that could be re-run against a local model to empirically measure the local/cloud capability gap for this specific *class* of task (infrastructure debugging via tool use), as distinct from the more commonly benchmarked class of task (coding, question-answering) — a useful axis for this research if not already covered.

11.1 Preliminary Local-Model Attempt (Qualitative)

The open question above was tested informally, not as a controlled experiment: the same class of debugging task was attempted with locally-hosted models from the research host's existing library (25 models, 975 GB, spanning macOS, `gb10`, and `amd-halo`), primarily:

- `qwen/qwen3.6-35b-a3b` (35B-A3B mixture-of-experts, `qwen35moe` architecture, ~22 GB) — the main model attempted.
- `agents-a1-nvfp4-mtp` (256×2.6B MoE, `qwen35moe` architecture, NVFP4-quantized with multi-token prediction, ~20.3 GB, branded/tuned specifically for agentic tool-use workloads) — made somewhat further progress than the above, but still did not resolve the task.

Neither model completed the diagnostic chain. Observed failure modes, consistent with each other across both models:

- **Hallucinated tool output** — claiming to have read a log, file, or command result that did not match what the tool actually returned.
- **Premature declaration of success** — reporting the issue fixed without having actually re-verified the fix against live behavior.
- **Looping on the same tool call** — repeating an identical command or check rather than revising the hypothesis when it failed to yield new information.

This is anecdotal, single-pass, and not a substitute for a rigorous comparison (no controlled prompt, no repeated trials, no scoring rubric), but it is directionally consistent with the open question in §11: this specific task class — sustained multi-hypothesis tracking, correlating asynchronous evidence (logs vs. live process state) over many tool-call rounds, and reading unfamiliar third-party source to extract a single load-bearing detail — appears to be where these particular local models, at these parameter counts and quantizations, currently fall down, even though they are otherwise capable of routine agentic tool use. A follow-up experiment worth running for the paper: repeat this exact diagnostic task (or a matched task of comparable structure, to avoid contamination from having the answer already documented) against each local model under identical tool access, and score against root-cause correctness, hallucination rate, and rounds-to-completion (or non-completion), to turn this qualitative observation into quantitative data.

11.2 A Confound: The Bootstrapping Problem

The comparison in §11.1 is not a clean, controlled one, for a structural reason worth naming explicitly

rather than glossing over: **the local models' tool access ran through the very MCP layer that was broken**. Asking a local model to diagnose its own missing/degraded tool access is, to a real degree, a bootstrapping problem — on macOS specifically, the `.zshrc` bug in §6 broke *every* MCP server, not just `searxng`, so a local model relying on LM Studio's MCP integration for tool access had, at that moment, little to no working external tool access at all: no web search to consult LM Studio/Electron/MCP documentation or known-issue reports, and depending on configuration, potentially no working filesystem tool either. The model needed functioning tool access to gather the evidence required to fix that same tool access.

The cloud-model harness used in §3–§10 (Claude Code) did not have this dependency: it supplies its own independent tool implementations (shell execution, file read/write, log inspection) entirely outside of, and unaffected by, LM Studio's MCP subsystem. Its ability to do this diagnostic work was never contingent on the thing being repaired.

This reframes the finding in §11.1: it is not only (or even primarily) evidence of a raw capability gap between model weights at inference time. It also reflects a difference in how tool access is typically *wired up* around each kind of model in practice — a cloud frontier model is usually reached through a dedicated agent harness with its own decoupled tool implementations, while a local model, in this setup, had its external tool access routed entirely through the single application subsystem under repair. That is a generalizable risk: any setup where a model's only path to tool access is a single integrated layer is vulnerable to exactly this kind of self-referential failure, regardless of how capable the underlying model is.

This confound was, in fact, already tested directly rather than left purely hypothetical: the same local models from §11.1 were also driven through `opencode`, a CLI agent harness that — like Claude Code — supplies its own tool implementations (shell execution, file read/write, log inspection) fully decoupled from LM Studio's MCP subsystem. This removes the bootstrapping confound described above: tool access was independent of the broken subsystem being diagnosed.

The result did not change the qualitative finding — if anything, it sharpened it. Even with unrestricted, decoupled tool access, the local models **could not** do what §3–§10 required: they could not reliably read log files on the fly and correlate them against live system state, could not study LM Studio's own (minified, third-party) source to locate a single load-bearing implementation detail, and could not reason through the `environment/npz/PATH` chain of issues in §4. This is stronger evidence than §11.1 alone that the gap is a genuine reasoning-capability gap at these model sizes and quantizations for this specific task class, rather than an artifact of routing tool access through a single integrated (and, at the time, broken) application layer. The tool-access-architecture point in this section remains

a real, separate risk worth designing around — but it is not, on this evidence, the primary explanation for the capability gap observed.

12. Safety Implications: Local-Model Agents Require Strict Reversibility Discipline

A separate, and arguably more operationally urgent, finding emerged from using `opencode` to give local models unrestricted tool access on these research systems: doing so is not merely a capability question but a **safety** one. In prior work on a different codebase (outside the scope of this document), local models driven with full free rein over the local system via an agent harness exhibited destructive behavior — unintended, damaging modifications made in the course of the model's own (incorrect) analysis or attempted fix.

This is a distinct axis from the reasoning-capability gap documented in §11: it is not just that these local models are less likely to *solve* a given problem correctly, but that an incorrect or confused local-model agent, given unrestricted tool access, can actively cause harm to the system it is operating on — deleting, overwriting, or corrupting state in the process of a failed attempt, rather than simply failing to produce a fix. A cloud-frontier-model harness (as used in §3–§10) is generally built with layered caution around exactly this class of action — e.g., explicit confirmation before destructive or hard-to-reverse operations, a preference for reversible steps, and checking repository state before any command that could discard work — but there is no guarantee a general-purpose harness paired with a weaker or less-aligned local model will exhibit, or reliably respect, the same judgment.

This exposes a critical operational requirement for any research (or production) use of local-model agents with real tool access: heavy, disciplined use of git branching, such that every local-model analysis or modification attempt occurs on a disposable branch and can be completely and cleanly reverted regardless of what the model actually did. This is not optional best practice in this context — it is a necessary compensating control, since the model's own judgment cannot be relied upon as the sole safeguard against irreversible damage. Practically, this means: a fresh branch (or disposable worktree/clone) per local-model agent session, no direct operation on a primary/shared branch, and treating the entire working tree as recoverable-but-not-trusted for the duration of any local-model-driven session.

This is worth foregrounding as a research finding in its own right, independent of the raw capability comparisons in §11: the practical tradeoff between local and cloud models is not only cost-vs-cost or reasoning-depth-vs-reasoning-depth, but also includes a **blast-radius** dimension — the operational

overhead required to safely contain a less reliable agent is itself part of the true cost of running local models autonomously.